Dokumentation – Wetter-Client (Open-Meteo API)

Erstellt von: *Roman Schesler*

Klasse: *E3FI2*

Projekt: *API-Abfrage und Analyse von Wetterdaten (Projekt Nr. 2)*

Datum: *13. März 2026*

Inhaltsverzeichnis

Dokumentation – Wetter-Client (Open-Meteo API)....1
1. Einleitung....2
2. Funktionsweise des Programms....2
2.1 Standortwahl & Benutzereingabe....2
2.2 Datenerhebung (HTTP-Request)....2
2.3 Verarbeitung & Analyse (JSON-Parsing)....2
3. Programmausführung & Analyse....3
3.1 Tägliche Wetterübersicht....3
3.2 Zusammenfassende Analyse....4
4. Grafische Darstellung....4
5. Datenspeicherung (Export)....4
6. Datenfluss und Architektur....5
7. Replikations-Leitfaden (Anleitung für Entwickler)....6
7.1 Voraussetzungen & Tools....6
7.2 Entwicklungsschritte....6
7.3 Erwartetes Ergebnis....6

1. Einleitung

Das Ziel dieses Projekts ist die Entwicklung eines Python-basierten Clients, der Wetterdaten in Echtzeit über eine REST-API abruft und auswertet.

Das Programm nutzt die **Open-Meteo API**, um sowohl eine stündliche Vorhersage als auch eine tägliche Zusammenfassung relevanter Wetterparameter zu liefern. Im Fokus stehen Höchst- und Tiefsttemperaturen, Niederschlagsmengen, Windgeschwindigkeiten sowie Sonnenauf- und -untergangszeiten.

Neben der reinen Datenausgabe werden die Werte automatisch analysiert (z. B. Erkennung von Frosttagen) und verständlich aufbereitet. Dadurch entsteht ein vollständiges Analyse-Tool, das über eine einfache Rohdaten-Ausgabe hinausgeht.

---

2. Funktionsweise des Programms

Der technische Ablauf des Programms gliedert sich in folgende Lernschritte:

2.1 Standortwahl & Benutzereingabe

Beim Programmstart wird der Benutzer durch ein Menü geführt. Zur Auswahl stehen drei vordefinierte deutsche Städte (Mannheim, Berlin, München) oder die manuelle Eingabe geografischer Koordinaten. Bei manueller Eingabe findet eine **Plausibilitätsprüfung** statt:

- **Breitengrad (lat):** -90° ≤ lat ≤ 90°
- **Längengrad (lon):** -180° ≤ lon ≤ 180°
  Ungültige Werte führen zu einer Fehlermeldung und einem kontrollierten Programmabbruch.

2.2 Datenerhebung (HTTP-Request)

- Über die Bibliothek requests wird ein **HTTP-GET-Request** an den Open-Meteo-Endpunkt gesendet.
- Die Parameter werden dynamisch aus der Benutzerauswahl übernommen.
- Alle Zeiten werden in der Zeitzone Europe/Berlin abgefragt; der Benutzer wird explizit darauf hingewiesen.

2.3 Verarbeitung & Analyse (JSON-Parsing)

Die Antwort des Servers wird in ein Python-Dictionary umgewandelt. Aus den daily-Daten werden Kenngrößen extrahiert und folgende Analysen berechnet:

- **Extremwerte:** Wärmster und kältester Tag im Zeitraum.
- **Frosttage:** Anzahl der Tage mit einer Minimaltemperatur unter 0°C.
- **Niederschlag:** Berechnung der Gesamtregenmenge und verbale Klassifikation.

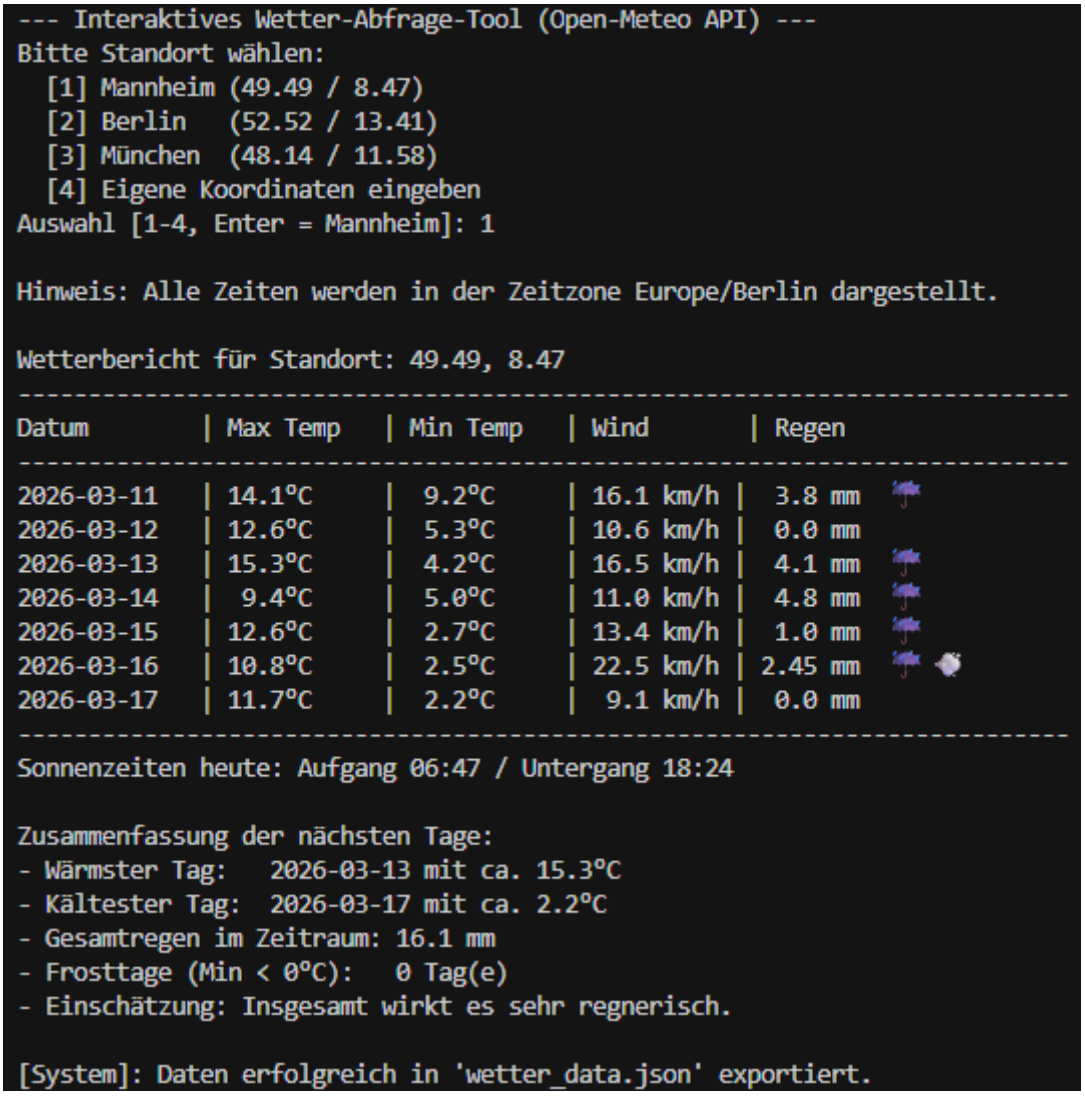
Schwellenwerte für die Regenklassifikation:

- **< 1 mm:** „eher trocken“ (kaum Regen)
- **1–10 mm:** „mäßig nass“ (moderater Regen)
- **> 10 mm:** „sehr regnerisch“ (anhaltender Niederschlag)

---

3. Programmausführung & Analyse

Abbildung 1: Konsolenausgabe mit interaktiver Standortwahl und Wetteranalyse



```
--- Interaktives Wetter-Abfrage-Tool (Open-Meteo API) ---
Bitte Standort wählen:
  [1] Mannheim (49.49 / 8.47)
  [2] Berlin   (52.52 / 13.41)
  [3] München  (48.14 / 11.58)
  [4] Eigene Koordinaten eingeben
Auswahl [1-4, Enter = Mannheim]: 1

Hinweis: Alle Zeiten werden in der Zeitzone Europe/Berlin dargestellt.

Wetterbericht für Standort: 49.49, 8.47
------------------------------------------------------------------------
Datum        | Max Temp   | Min Temp   | Wind       | Regen
------------------------------------------------------------------------
2026-03-11   | 14.1°C     |  9.2°C     | 16.1 km/h |  3.8 mm
2026-03-12   | 12.6°C     |  5.3°C     | 10.6 km/h |  0.0 mm
2026-03-13   | 15.3°C     |  4.2°C     | 16.5 km/h |  4.1 mm
2026-03-14   |  9.4°C     |  5.0°C     | 11.0 km/h |  4.8 mm
2026-03-15   | 12.6°C     |  2.7°C     | 13.4 km/h |  1.0 mm
2026-03-16   | 10.8°C     |  2.5°C     | 22.5 km/h | 2.45 mm
2026-03-17   | 11.7°C     |  2.2°C     |  9.1 km/h |  0.0 mm
------------------------------------------------------------------------
Sonnenzeiten heute: Aufgang 06:47 / Untergang 18:24

Zusammenfassung der nächsten Tage:
- Wärmster Tag:   2026-03-13 mit ca. 15.3°C
- Kältester Tag:  2026-03-17 mit ca. 2.2°C
- Gesamtregen im Zeitraum: 16.1 mm
- Frosttage (Min < 0°C):   0 Tag(e)
- Einschätzung: Insgesamt wirkt es sehr regnerisch.

[System]: Daten erfolgreich in 'wetter_data.json' exportiert.
```

3.1 Tägliche Wetterübersicht

Das Programm erzeugt eine Tabelle mit Icons zur schnellen Orientierung:

- ❄️ für Frosttage
- ☂️ für Tage mit Niederschlag
- 💨 für Tage mit starkem Wind

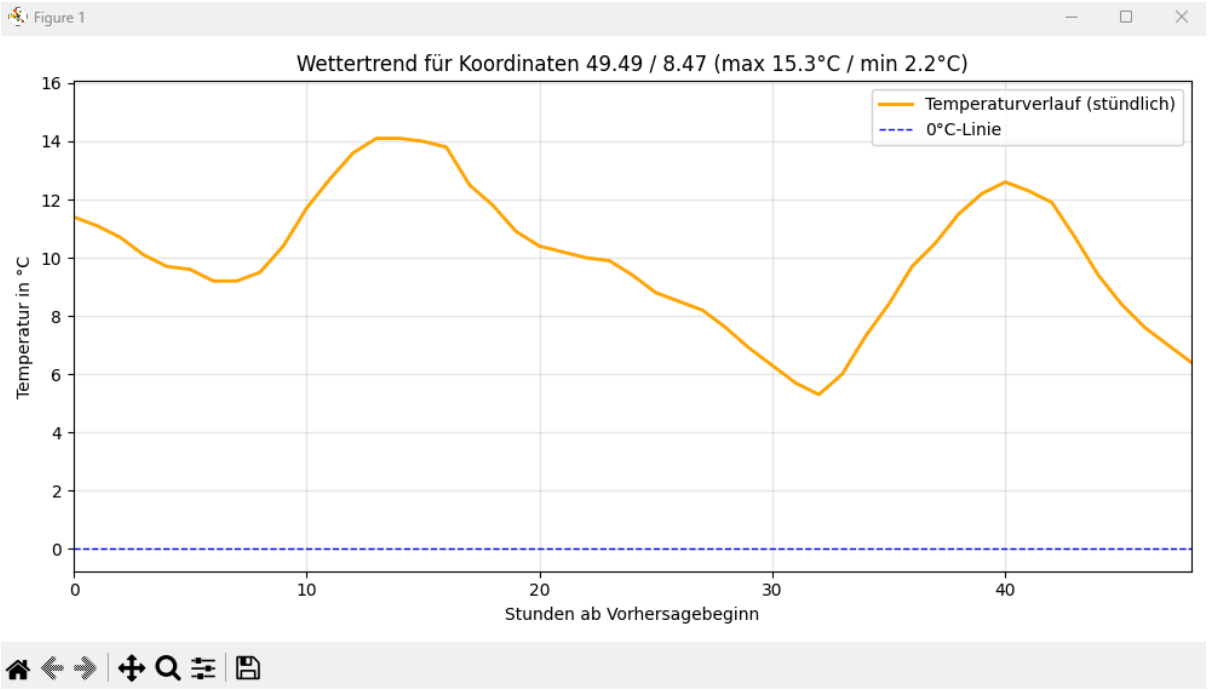
3.2 Zusammenfassende Analyse

Unterhalb der Tabelle wird eine automatische Zusammenfassung ausgegeben, die den wärmsten/kältesten Tag sowie die Regen-Einschätzung enthält. Dies bietet eine direkte Interpretation der Daten für den Endnutzer.

---

4. Grafische Darstellung

Abbildung 2: Visualisierung des stündlichen Temperaturverlaufs



Zur Veranschaulichung wird ein Diagramm des stündlichen Temperaturverlaufs (Matplotlib) erstellt:

- **Orangefarbene Linie:** Temperaturverlauf der kommenden Stunden.
- **Blaue gestrichelte Linie:** Markierung des Gefrierpunkts (0°C).
- **Diagrammtitel:** Anzeige der Koordinaten sowie der absoluten Höchst- und Tiefstwerte.
- Die x-Achse visualisiert maximal 48 Stunden für eine optimale Übersicht.

---

5. Datenspeicherung (Export)

Bei jedem Programmlauf wird eine Datei namens wetter_data.json erzeugt. Der Export umfasst eine kuratierte Struktur:

- **request:** Verwendete Parameter (Koordinaten, Zeitzone).

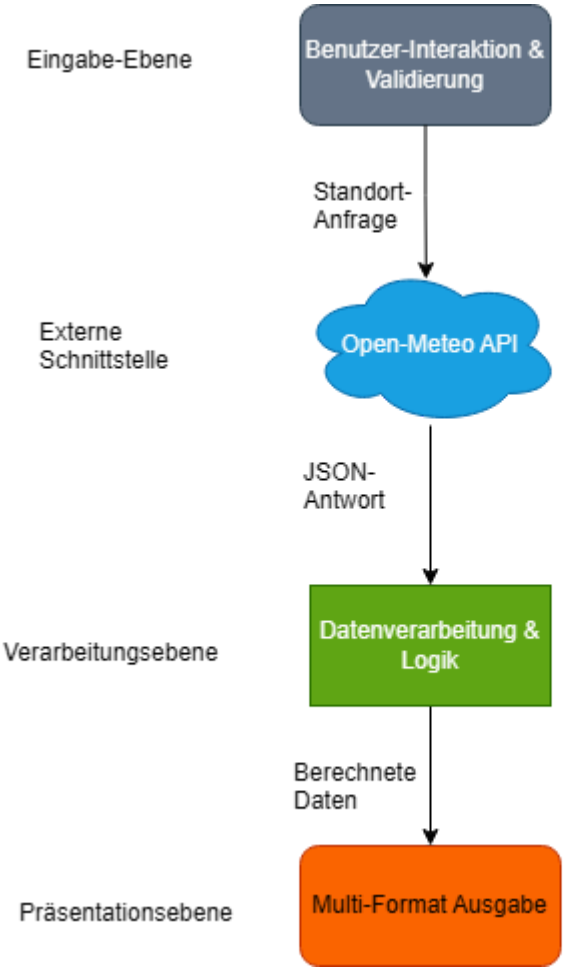
- **daily / hourly:** Die extrahierten Zeitreihen der API.
- **analysis:** Alle berechneten Ergebnisse (Frosttage, Extrema, Regen-Klasse).

Die Datei ist UTF-8-kodiert und eingerückt (indent=4), was sowohl die Maschinenlesbarkeit als auch die manuelle Prüfung erleichtert.

---

6. Datenfluss und Architektur

Abbildung 3: Datenflussdiagramm (Data Flow Diagram, DFD)



Die Architektur folgt einer klaren Trennung der Zuständigkeiten:

1. **Benutzer:** Eingabe/Wahl des Standorts.
2. **Python-Client:** Validierung und Erstellung des Requests.
3. **Open-Meteo API:** Bereitstellung der Wetterdaten (JSON).
4. **Parsing & Analyse:** Logische Verarbeitung im Client.
5. **Ausgabe & Speicherung:** Darstellung (Konsole/Graph) und Datei-Export.

Dieses Modell unterstreicht die Robustheit des Programms und die strukturierte Verarbeitung von der Eingabe bis zum Ergebnis.

7. Replikations-Leitfaden (Anleitung für Entwickler)

Dieser Abschnitt dient als Orientierungshilfe, um das Projekt eigenständig nachzubauen oder zu erweitern.

7.1 Voraussetzungen & Tools

Um das Projekt erfolgreich umzusetzen, müssen folgende Werkzeuge installiert sein:

- ***Python 3.x:*** *Die Basis-Programmiersprache.*
- ***Bibliotheken:*** *Installation über das Terminal: pip install requests pandas matplotlib*
- ***API-Quelle:*** *Open-Meteo (kein API-Key erforderlich).*

7.2 Entwicklungsschritte

Ein neuer Entwickler sollte die Implementierung in dieser Reihenfolge durchführen:

1. ***Menü-Struktur:*** *Erstellung einer Auswahl für Städte inklusive einer manuellen Koordinateneingabe.*
2. ***Validierung:*** *Einbau einer if-else- oder try-except-Logik, um sicherzustellen, dass Breitengrade zwischen -90° / 90° und Längengrade zwischen -180° / 180° liegen.*
3. ***API-Abfrage:*** *Nutzung der requests-Library, um Daten im JSON-Format abzurufen (wichtig: Parameter timezone=Europe/Berlin setzen).*
4. ***Datenverarbeitung:*** *Extraktion der daily- und hourly-Werte aus dem empfangenen JSON-Dictionary.*
5. ***Analyse-Funktionen:*** *Programmierung der Logik für Frosttage (Min-Temperatur < 0°C) und die verbale Regen-Klassifizierung.*
6. ***Visualisierung:*** *Nutzung von matplotlib zur Erstellung des Temperaturverlaufs (inklusive einer markanten 0°C-Linie).*
7. ***Daten-Export:*** *Speicherung der finalen Ergebnisse in einer wetter_data.json mithilfe von json.dump().*

7.3 Erwartetes Ergebnis

Nach erfolgreicher Umsetzung muss das Programm eine formatierte Tabelle in der Konsole ausgeben, ein grafisches Fenster mit dem Temperaturplot öffnen und eine strukturierte Export-Datei erstellen.

Die Architektur des Programms wurde bewusst modular aufgebaut, sodass zukünftige Erweiterungen (z. B. weitere Städte oder zusätzliche Wetterparameter) problemlos integriert werden können.